

HookChain

A new perspective for Bypassing EDR Solutions

Helvio Junior (M4v3r1ck)



Agenda

- Whoami
- What is HookChain?
 - Research limitations
 - Protection rings
 - Windows APIs and process loading
 - Function Hooks
 - EDR overview
 - Most common EDR bypass public techniques
 - HookChain

Whoami

- Helvio Junior (M4v3r1ck)
- Airplane pilot
- First to earn OSCE³ in Latin America
- Studying to OSEE
- Research topics:
 - Low Level Security
 - Buffer Overflow
 - Shellcoding
 - Malware development
 - AV/EDR/XDR Bypasses
 - Mobile and etc...
- CEO of Sec4US



Motivation

- Many people believe, I have **solution X**, which is a Gartner leader, so I am 100% protected
- There is no silver bullet
- Bypass/avoidance of defense layers
- **FUD** - **F**ully **U**n**D**etectable
- FUD is when your code is not detected by any defense layer/tool
- It is important to be 100% FUD regarding your target

What is HookChain?

- ✓ Is a Technique
- ✓ It is NOT a tool
- ✓ User-land NTDLL Hook bypass





Limitations!

- This study aims to demonstrate a new bypass technique using the interception of the operating system API functions of Microsoft Windows© 64-bit in **user mode**.
- Limited just at Ntdll hooks bypass.
- This does **NOT** aim to classify the defense and EDR products demonstrated here in terms of their effectiveness, efficacy, and quality in the process of protecting and defending the assets where they are installed, as it is a study and presentation of a technique focused on a single point of identification of the agents.

Ethics!



- This study does not represent ethical violations
- All tests were conducted in controlled environments with valid licensing
- I called several EDR/XDR vendors to have early access at this research and PoC, but **no one answered**
- https://www.linkedin.com/posts/helviojunior_hookchain-edrbypass-xdrbypass-activity-7188225698434596865-3Jxy
- The public released code is focused at HookChain itself

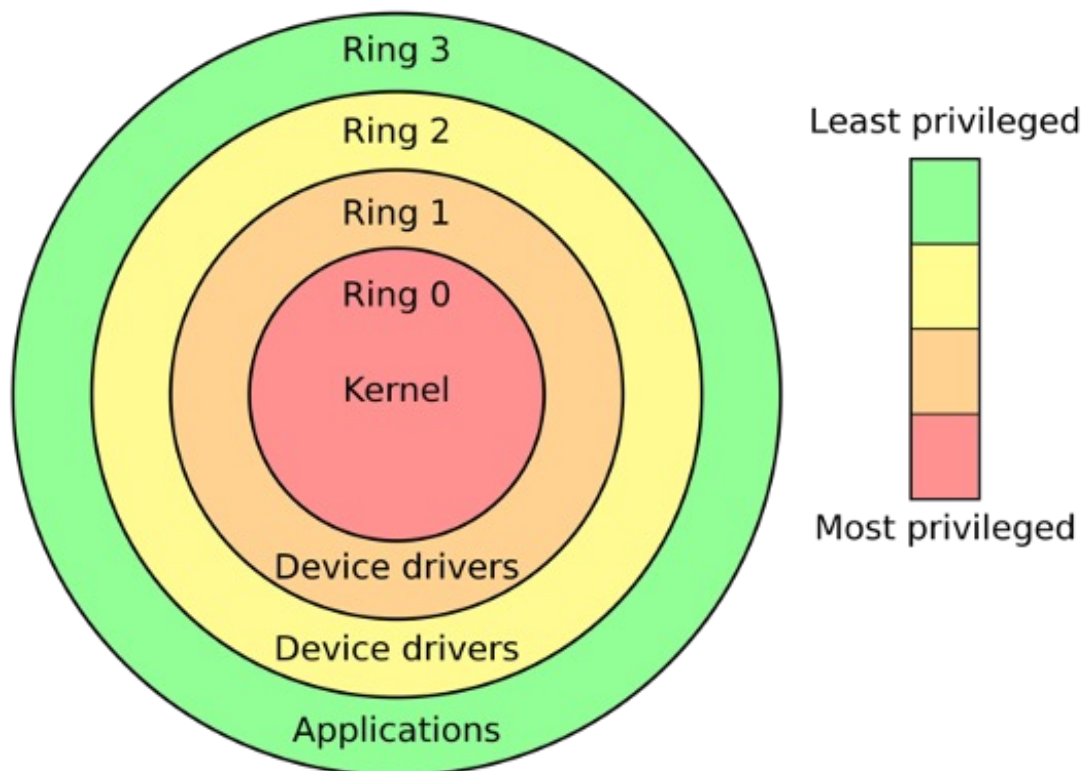
Protection Rings

From a security point of view, there are protection rings

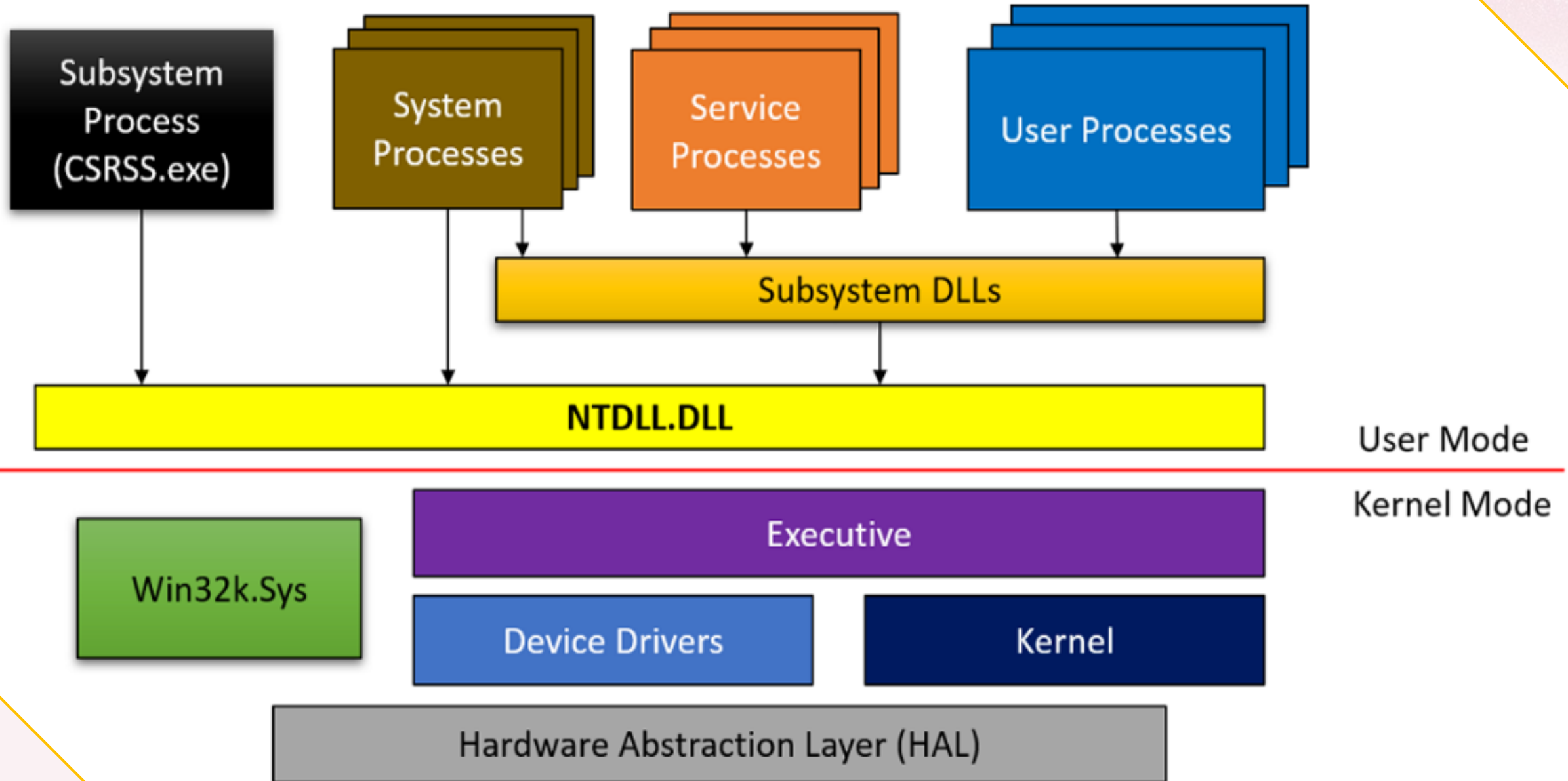
Aims to create protection mechanisms against failures and malicious actions

For today's topic, the main objective is to protect critical areas of the OS from applications executed by the user

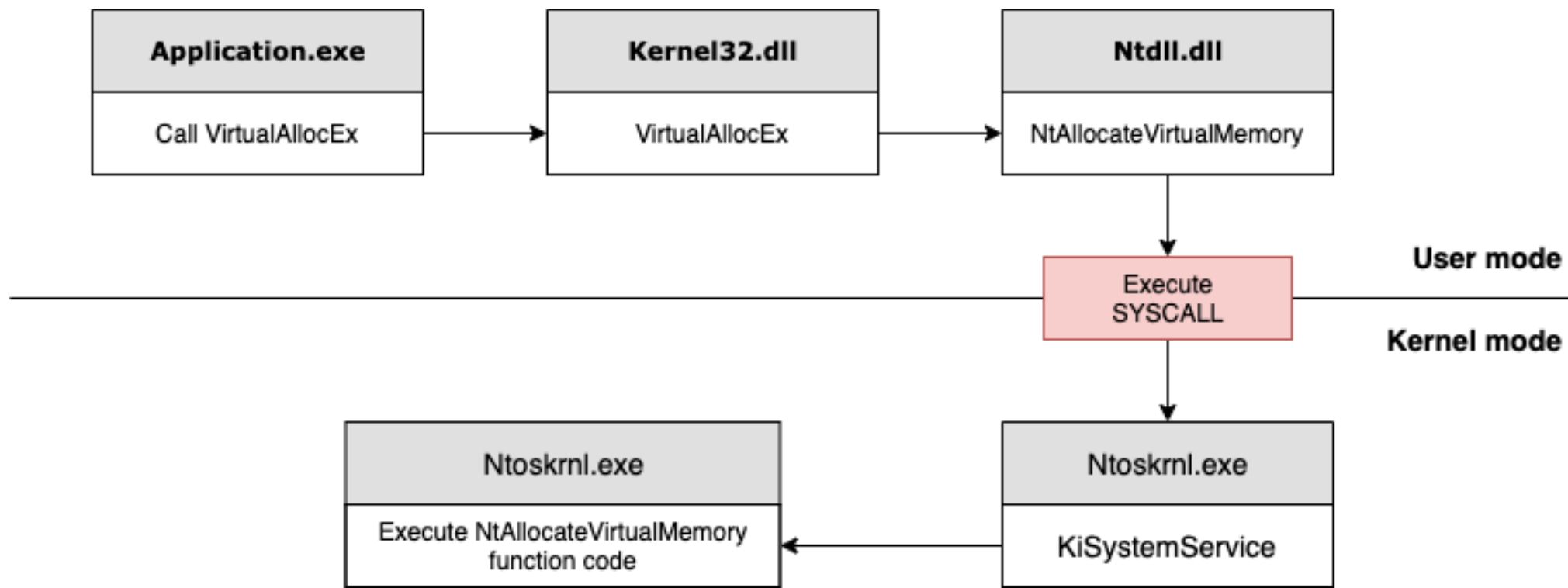
- **Ring 3** (user mode/user land): Restricted access to resources and
- **Ring 0** (kernel mode): Has direct access to resources such as memory, CPU, system, etc.



Windows APIs

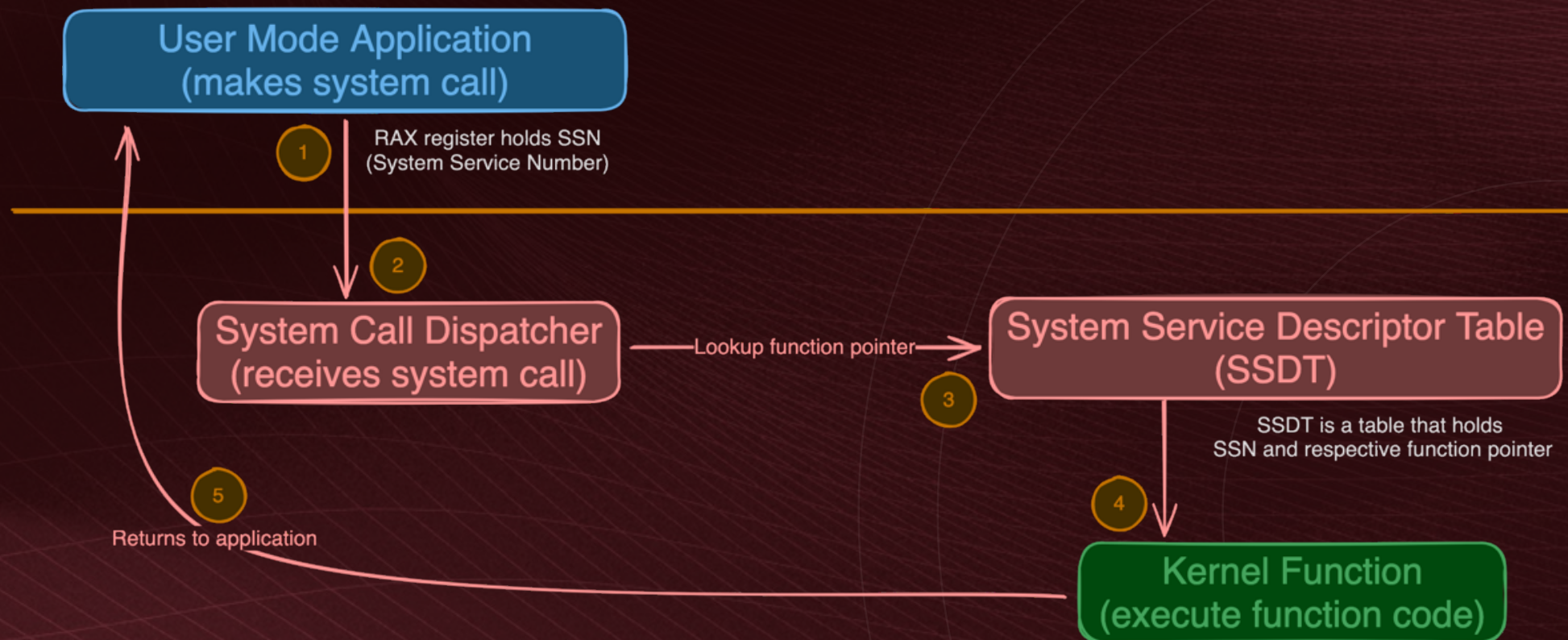


Common call stack



Common execution flow

- The user mode application call **SYSCALL** assembly instruction
- The system call dispatcher uses the system call number (**SSN**) to look up the corresponding function pointer in the SSDT.
- The appropriate kernel function is called.
- After the kernel function completes, control is returned to the user mode application.



Common execution flow

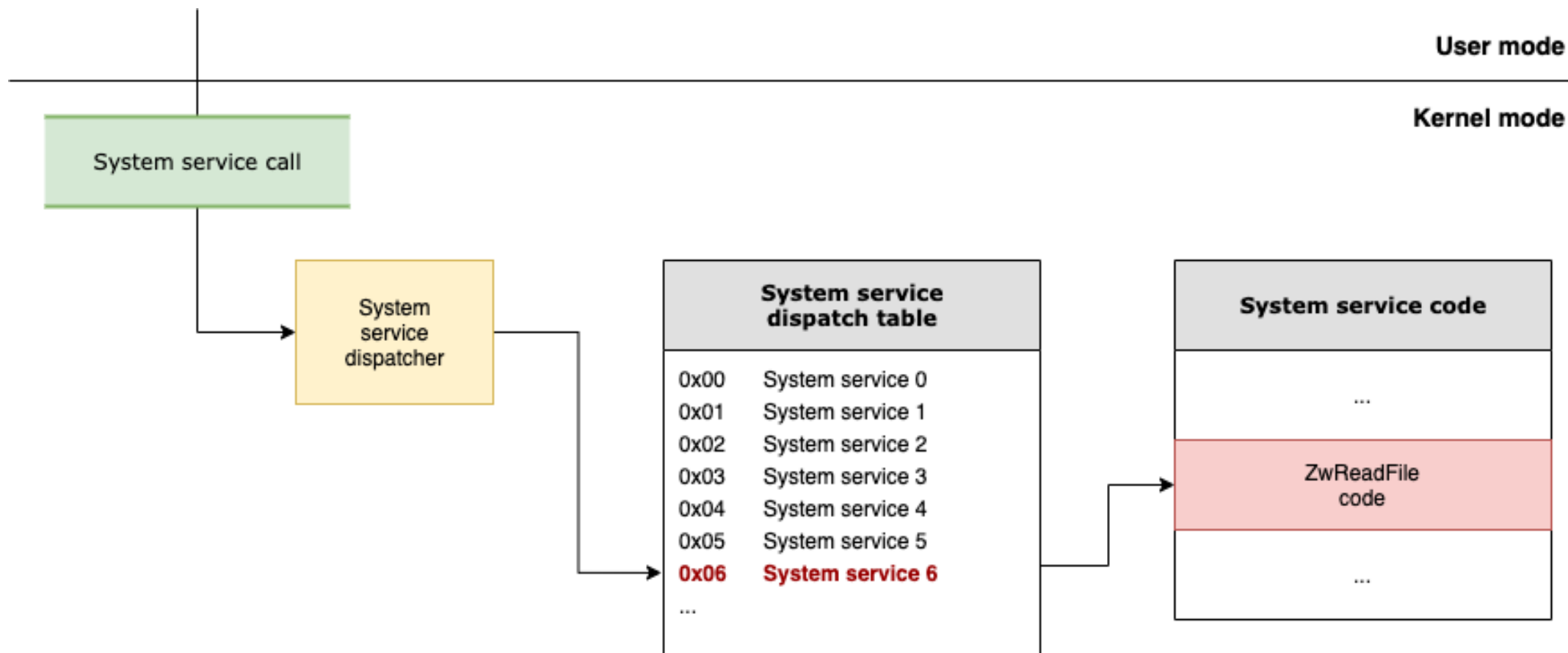


Image Loader

- ✓ When a process is initiated, or a new DLL have been loaded
- ✓ Several actions are carried out internally by the operating system
- ✓ The image loader is a user-mode resident code, within Ntdll.dll and not in a kernel library
- ✓ Import Address Table (IAT) is filled with the current addresses of the function in memory

Image Loader

IAT - Import Address Table

DLL

Loads each one of the DLLs referenced in the PE import table.

Previous loaded

Checks if the DLL in question is already loaded into the process's memory, if not, reads the DLL from disk and maps it into memory.

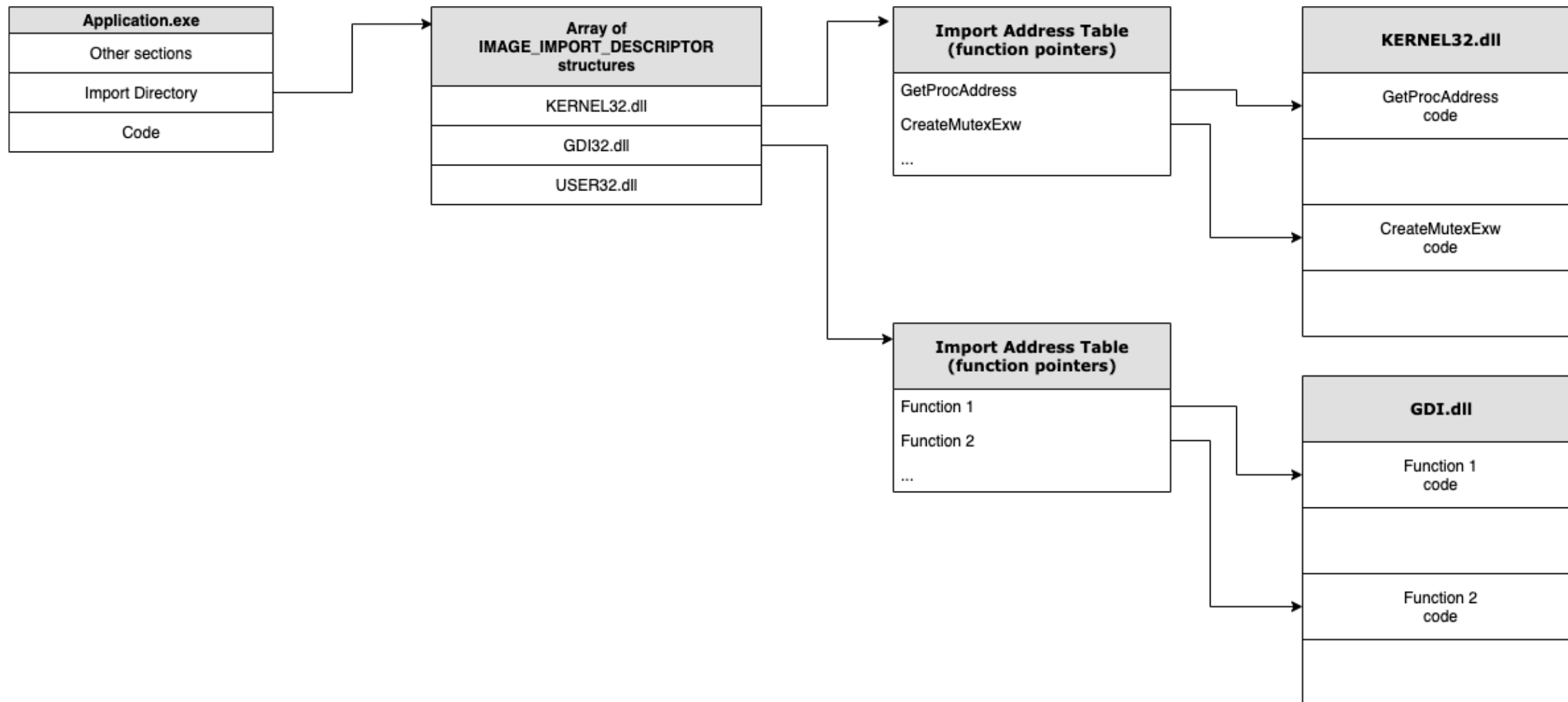
Recursive loading

After mapping the DLL into memory, this process is repeated for this DLL with the goal of importing the dependencies used by it.

Fill IAT with function pointer

After each DLL is loaded, the IAT is processed looking for the specific functions to be imported. For each imported name, the loader checks the export table of the imported DLL and tries to locate the desired function.

PE Import Schema



Function Hook

- Function interception (Hook) is not something new and has various applications such as application debugging. But also, in the monitoring process by defense layers (EDR/XDR).
- Let's take a look at 2 types:
 - JMP or Call
 - IAT Hook

Function Hook

Ntdll JMP

notepad.exe - PID: 804 - Module: ntdll.dll - Thread: Main Thread 11476 - x64dbg

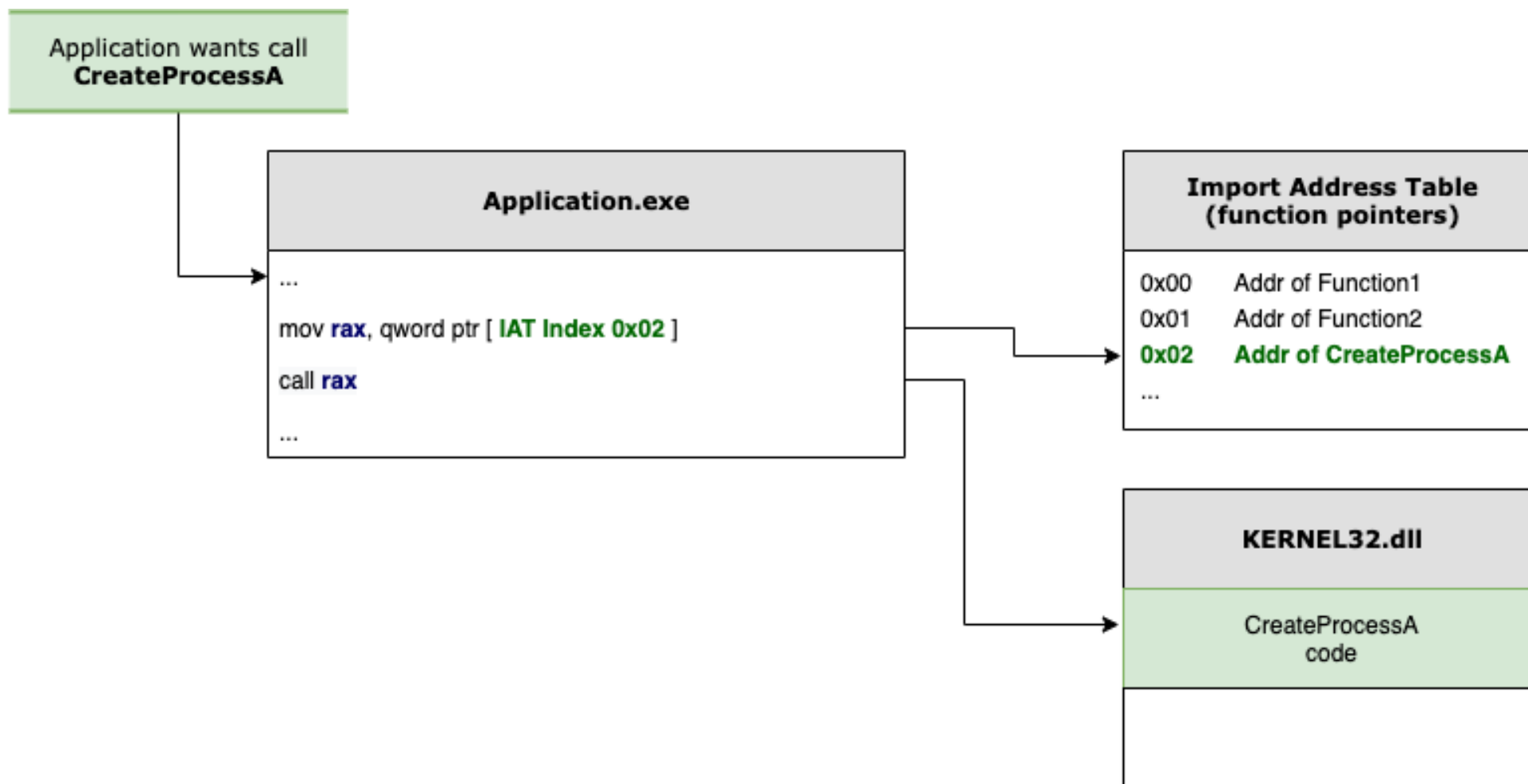
File View Debug Tracing Plugins Favourites Options Help Jul 19 2023 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads Handles

00007FF93840D6F0	4C:8BD1	mov r10,rcx	ZwFsControlFile
00007FF93840D6F3	B8 39000000	mov eax,39	39: '9'
00007FF93840D6F8	F60425 0803FE7F 01	test byte ptr ds:[7FFE0308],1	
00007FF93840D700	75 03	jne ntdll.7FF93840D705	
00007FF93840D702	0F05	syscall	
00007FF93840D704	C3	ret	
00007FF93840D705	CD 2E	int 2E	
00007FF93840D707	C3	ret	
00007FF93840D708	0F1F8400 00000000	nop dword ptr ds:[rax+rax],eax	
00007FF93840D710	E9 8B311600	jmp 7FF9385708A0	NtWriteVirtualMemory
00007FF93840D715	CC	int3	
00007FF93840D716	CC	int3	
00007FF93840D717	CC	int3	
00007FF93840D718	F60425 0803FE7F 01	test byte ptr ds:[7FFE0308],1	
00007FF93840D720	75 03	jne ntdll.7FF93840D725	
00007FF93840D722	0F05	syscall	
00007FF93840D724	C3	ret	
00007FF93840D725	CD 2E	int 2E	
00007FF93840D727	C3	ret	
00007FF93840D728	0F1F8400 00000000	nop dword ptr ds:[rax+rax],eax	
00007FF93840D730	4C:8BD1	mov r10,rcx	NtCloseObjectAuditAlarm
00007FF93840D733	B8 38000000	mov eax,38	38: ';'
00007FF93840D738	F60425 0803FE7F 01	test byte ptr ds:[7FFE0308],1	
00007FF93840D740	75 03	jne ntdll.7FF93840D745	
00007FF93840D742	0F05	syscall	
00007FF93840D744	C3	ret	
00007FF93840D745	CD 2E	int 2E	
00007FF93840D747	C3	ret	
00007FF93840D748	0F1F8400 00000000	nop dword ptr ds:[rax+rax],eax	

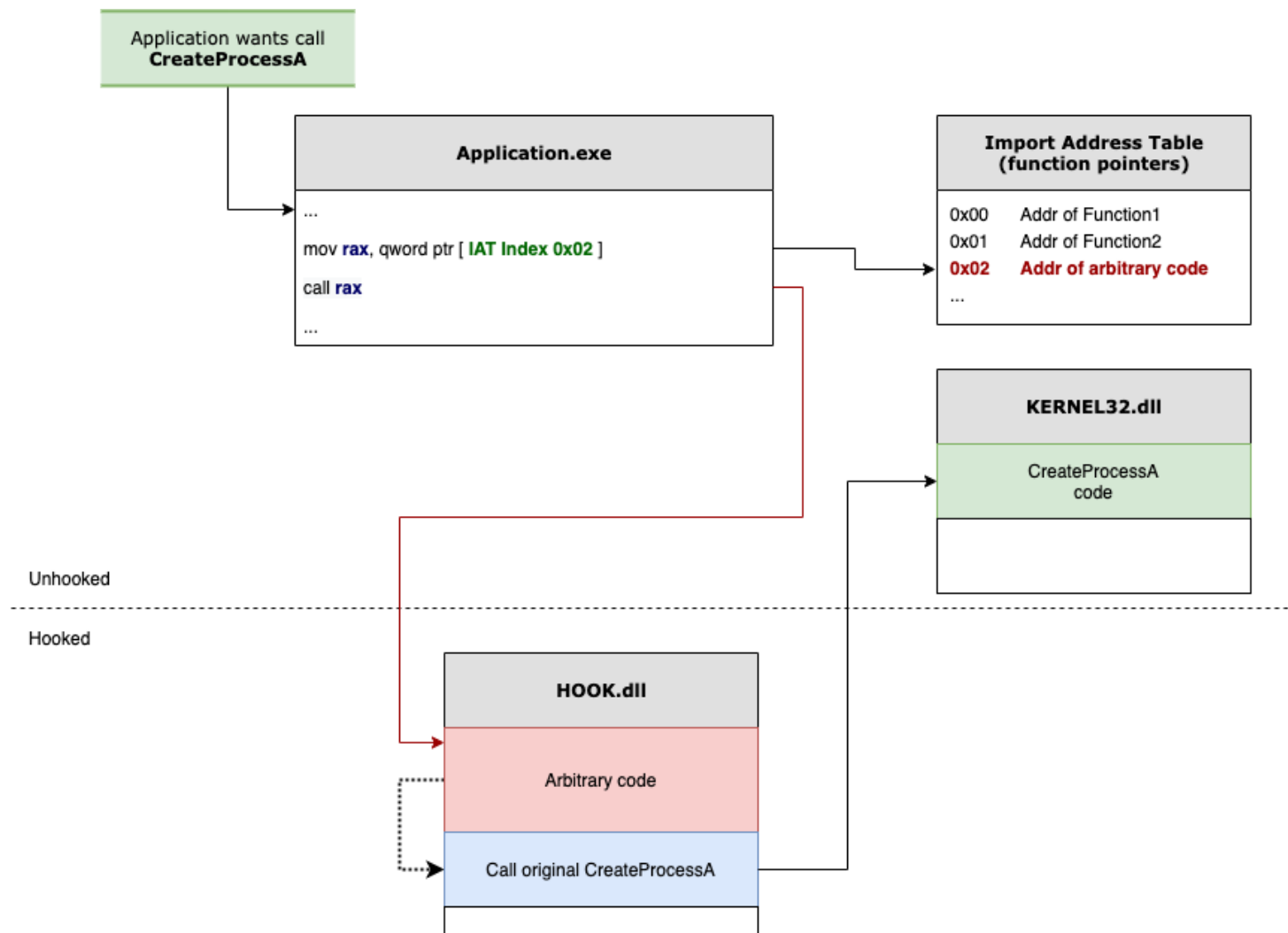
Function Hook

IAT Original/unhooked execution flow

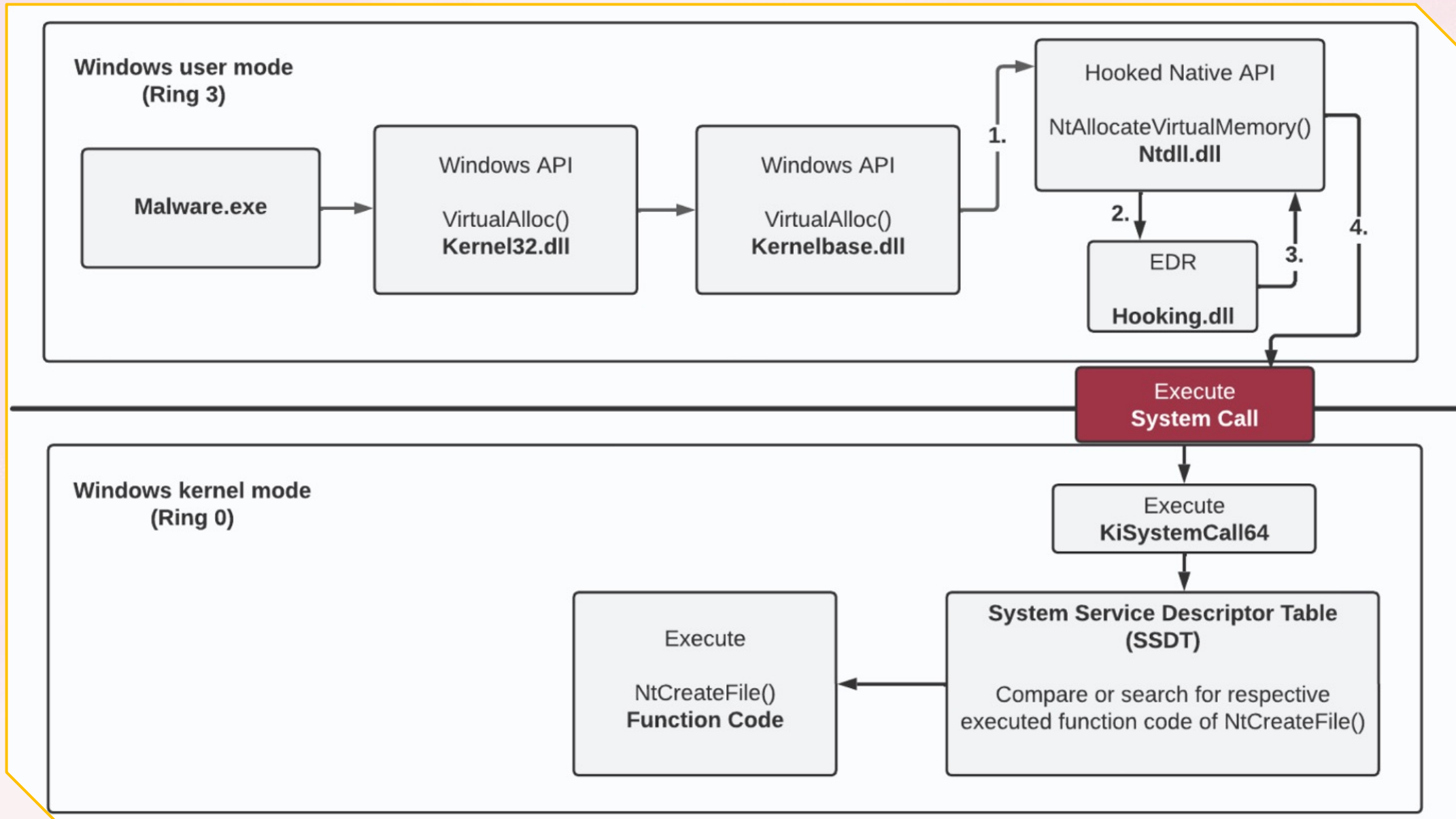


Function Hook

IAT Hooked execution flow



EDR Overview



Unhooked API

notepad.exe - P 1596 Module: ntdll.dll Thread: Main Thread 11500 x64dbg 1

File View Debug Tracing Plugins Favourites Options Help Sep 2 2023 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source

Base	Module	Address	Type	Ordinal	Symbol
00007FFC3E4F0000	ntdll.dll 2	00007FFC3E58FB20	Export	430	NtOpenProcess 3
		00007FFC3E58FC60	Export	432	NtOpenProcessTokenEx
		00007FFC3E591A90	Export	431	NtOpenProcessToken



Hooked API

notepad.exe - PID: 8004 - Module: ntdll.dll - Thread: Main Thread 8488 - x64dbg

File View Debug Tracing Plugins Favourites Options Help Sep 2 2023 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threa

Address	Disassembly	Comment
00007FFEDE5AFB00	4C:8BD1	
00007FFEDE5AFB03	B8 25000000	
00007FFEDE5AFB08	F60425 0803FE7F 01	
00007FFEDE5AFB10	75 03	
00007FFEDE5AFB12	0F05	
00007FFEDE5AFB14	C3	
00007FFEDE5AFB15	CD 2E	
00007FFEDE5AFB17	C3	
00007FFEDE5AFB18	0F1F8400 00000000	
00007FFEDE5AFB20	E9 D6091500	
00007FFEDE5AFB25	CC	
00007FFEDE5AFB26	CC	
00007FFEDE5AFB27	CC	
00007FFEDE5AFB28	F60425 0803FE7F 01	
00007FFEDE5AFB30	75 03	
00007FFEDE5AFB32	0F05	
00007FFEDE5AFB34	C3	
00007FFEDE5AFB35	CD 2E	
00007FFEDE5AFB37	C3	
00007FFEDE5AFB38	0F1F8400 00000000	
00007FFEDE5AFB40	4C:8BD1	
00007FFEDE5AFB43	B8 27000000	
00007FFEDE5AFB48	F60425 0803FE7F 01	
00007FFEDE5AFB50	75 03	
00007FFEDE5AFB52	0F05	
00007FFEDE5AFB54	C3	
00007FFEDE5AFB55	CD 2E	
00007FFEDE5AFB57	C3	
00007FFEDE5AFB58	0F1F8400 00000000	
00007FFEDE5AFB60	E9 2E081500	
00007FFEDE5AFB65	CC	
00007FFEDE5AFB66	CC	
00007FFEDE5AFB67	CC	
00007FFEDE5AFB68	F60425 0803FE7F 01	
00007FFEDE5AFB70	75 03	
00007FFEDE5AFB72	0F05	
00007FFEDE5AFB74	C3	
00007FFEDE5AFB75	CD 2E	

mov r10,rcx
mov eax,25
test byte ptr ds:[7FFE0308],1
jne ntdll.7FFEDE5AFB15
syscall
ret
int 2E
ret
nop dword ptr ds:[rax+rax],eax
jmp 7FFEDE7004F8
int3
int3
int3
test byte ptr ds:[7FFE0308],1
jne ntdll.7FFEDE5AFB35
syscall
ret
int 2E
ret
nop dword ptr ds:[rax+rax],eax
mov r10,rcx
mov eax,27
test byte ptr ds:[7FFE0308],1
jne ntdll.7FFEDE5AFB55
syscall
ret
int 2E
ret
nop dword ptr ds:[rax+rax],eax
jmp 7FFEDE700693
int3
int3
int3
test byte ptr ds:[7FFE0308],1
jne ntdll.7FFEDE5AFB75
syscall
ret
int 2E

ZwQueryInformationThread
25: !!!
Unhooked

ZwOpenProcess
Hooked
ID 26

ZwSetInformationFile
27: !!!
Unhooked

ZwMapViewOfSection

Most Common Bypasses

Remapping of Ntdll.dll

Direct Syscalls

Indirect Syscall

Known techniques

Heaven's Gate

Developed by Roy G. Biv

Hell's Gate

Developed by smelly__vx (@RtlMateusz) and am0nsec (@am0nsec)

Halo's Gate

Developed by Reenz0h from Sektor7

Known techniques

Heaven's Gate

Developed by Roy G. Biv

Hell's Gate

Developed by smelly__vx (@RtlMateusz) and am0nsec (@am0nsec)

Halo's Gate

Developed by Reenz0h from Sektor7

Locates the current address of the desired function within the Ntdll.dll

If these are not the bytes, the function is being monitored (in other words, it has a Hook set), however, the neighboring functions (before and after) may not have a Hook.



Performs the reading of the function's bytes (currently 32 bytes) and checks if the function's bytes match those of the assembly instructions (mov r10, rcx; mov eax, SSN)

Searches in the neighboring functions (above and below) for functions without a Hook, and calculates the distance of the located function from the current function, thus having the current function's SSN code.

System Service Number - SSN

notepad.exe - PID: 1596 - Module: ntdll.dll - Thread: Main Thread 11500 - x64dbg

File View Debug Tracing Plugins Favourites Options Help Sep 2 2023 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Thread

Address	Disassembly	Comment
00007FFC3E58F660	4C:8BD1	
00007FFC3E58F663	B8 00000000	
00007FFC3E58F668	F60425 0803FE7F 01	
00007FFC3E58F670	75 03	
00007FFC3E58F672	0F05	
00007FFC3E58F674	C3	
00007FFC3E58F675	CD 2E	
00007FFC3E58F677	C3	
00007FFC3E58F678	0F1F8400 00000000	
00007FFC3E58F680	4C:8BD1	
00007FFC3E58F683	B8 01000000	
00007FFC3E58F688	F60425 0803FE7F 01	
00007FFC3E58F690	75 03	
00007FFC3E58F692	0F05	
00007FFC3E58F694	C3	
00007FFC3E58F695	CD 2E	
00007FFC3E58F697	C3	
00007FFC3E58F698	0F1F8400 00000000	
00007FFC3E58F6A0	4C:8BD1	
00007FFC3E58F6A3	B8 02000000	
00007FFC3E58F6A8	F60425 0803FE7F 01	
00007FFC3E58F6B0	75 03	
00007FFC3E58F6B2	0F05	
00007FFC3E58F6B4	C3	
00007FFC3E58F6B5	CD 2E	
00007FFC3E58F6B7	C3	
00007FFC3E58F6B8	0F1F8400 00000000	
00007FFC3E58F6C0	4C:8BD1	
00007FFC3E58F6C3	B8 03000000	
00007FFC3E58F6C8	F60425 0803FE7F 01	
00007FFC3E58F6D0	75 03	
00007FFC3E58F6D2	0F05	
00007FFC3E58F6D4	C3	
00007FFC3E58F6D5	CD 2E	
00007FFC3E58F6D7	C3	
00007FFC3E58F6D8	0F1F8400 00000000	
00007FFC3E58F6E0	4C:8BD1	
00007FFC3E58F6E3	B8 04000000	

mov r10,rcx
mov eax,0
test byte ptr ds:[7FFE0308],1
jne ntdll.7FFC3E58F675
syscall
ret
int 2E
ret
nop dword ptr ds:[rax+rax],eax
mov r10,rcx
mov eax,1
test byte ptr ds:[7FFE0308],1
jne ntdll.7FFC3E58F695
syscall
ret
int 2E
ret
nop dword ptr ds:[rax+rax],eax
mov r10,rcx
mov eax,2
test byte ptr ds:[7FFE0308],1
jne ntdll.7FFC3E58F6B5
syscall
ret
int 2E
ret
nop dword ptr ds:[rax+rax],eax
mov r10,rcx
mov eax,3
test byte ptr ds:[7FFE0308],1
jne ntdll.7FFC3E58F6D5
syscall
ret
int 2E
ret
nop dword ptr ds:[rax+rax],eax
mov r10,rcx
mov eax,4

ZwAccessCheck
NtworkerFactoryWorkerReady
ZwAcceptConnectPort
NtMapUserPhysicalPagesScatter
NtwaitForSingleObject

System Service Number – SSN (Hallo's Gate)

notepad.exe - PID: 6148 - Module: ntdll.dll - Thread: Main Thread 9220 - x64dbg

File View Debug Tracing Plugins Favourites Options Help Jul 19 2023 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads Handles Trace

Address	Disassembly	Comment
00007FFF62E4D090	mov r10,rcx	NtReadFile
00007FFF62E4D093	mov eax,6	
00007FFF62E4D098	test byte ptr ds:[7FFE0308],1	
00007FFF62E4D0A0	jne ntdll.7FFF62E4D0A5	
00007FFF62E4D0A2	syscall	
00007FFF62E4D0A4	ret	
00007FFF62E4D0A5	int 2E	
00007FFF62E4D0A7	ret	
00007FFF62E4D0A8	ret	
00007FFF62E4D0B0	nop dword ptr ds:[rax+rax],eax	ZwDeviceIoControlFile
00007FFF62E4D0B3	mov r10,rcx	
00007FFF62E4D0B8	mov eax,7	
00007FFF62E4D0C0	test byte ptr ds:[7FFE0308],1	
00007FFF62E4D0C2	jne ntdll.7FFF62E4D0C5	
00007FFF62E4D0C4	syscall	
00007FFF62E4D0C7	ret	
00007FFF62E4D0C8	ret	
00007FFF62E4D0D0	nop dword ptr ds:[rax+rax],eax	NtWriteFile
00007FFF62E4D0D5	jmp 7FFF62FB1D40	
00007FFF62E4D0D6	int3	
00007FFF62E4D0D7	int3	
00007FFF62E4D0D8	int3	
00007FFF62E4D0E0	test byte ptr ds:[7FFE0308],1	
00007FFF62E4D0E2	jne ntdll.7FFF62E4D0E5	
00007FFF62E4D0E4	syscall	
00007FFF62E4D0E5	ret	
00007FFF62E4D0E7	ret	
00007FFF62E4D0E8	ret	
00007FFF62E4D0F0	nop dword ptr ds:[rax+rax],eax	ZwRemoveIoCompletion
00007FFF62E4D0F3	mov r10,rcx	
00007FFF62E4D0F8	mov eax,9	
00007FFF62E4D100	test byte ptr ds:[7FFE0308],1	
00007FFF62E4D102	jne ntdll.7FFF62E4D105	
00007FFF62E4D104	syscall	
00007FFF62E4D107	ret	
00007FFF62E4D108	ret	
00007FFF62E4D110	nop dword ptr ds:[rax+rax],eax	NtReleaseSemaphore
00007FFF62E4D113	mov r10,rcx	
00007FFF62E4D118	mov eax,A	
00007FFF62E4D120	test byte ptr ds:[7FFE0308],1	
00007FFF62E4D122	jne ntdll.7FFF62E4D125	
00007FFF62E4D124	syscall	
00007FFF62E4D125	ret	
00007FFF62E4D127	ret	
00007FFF62E4D128	ret	

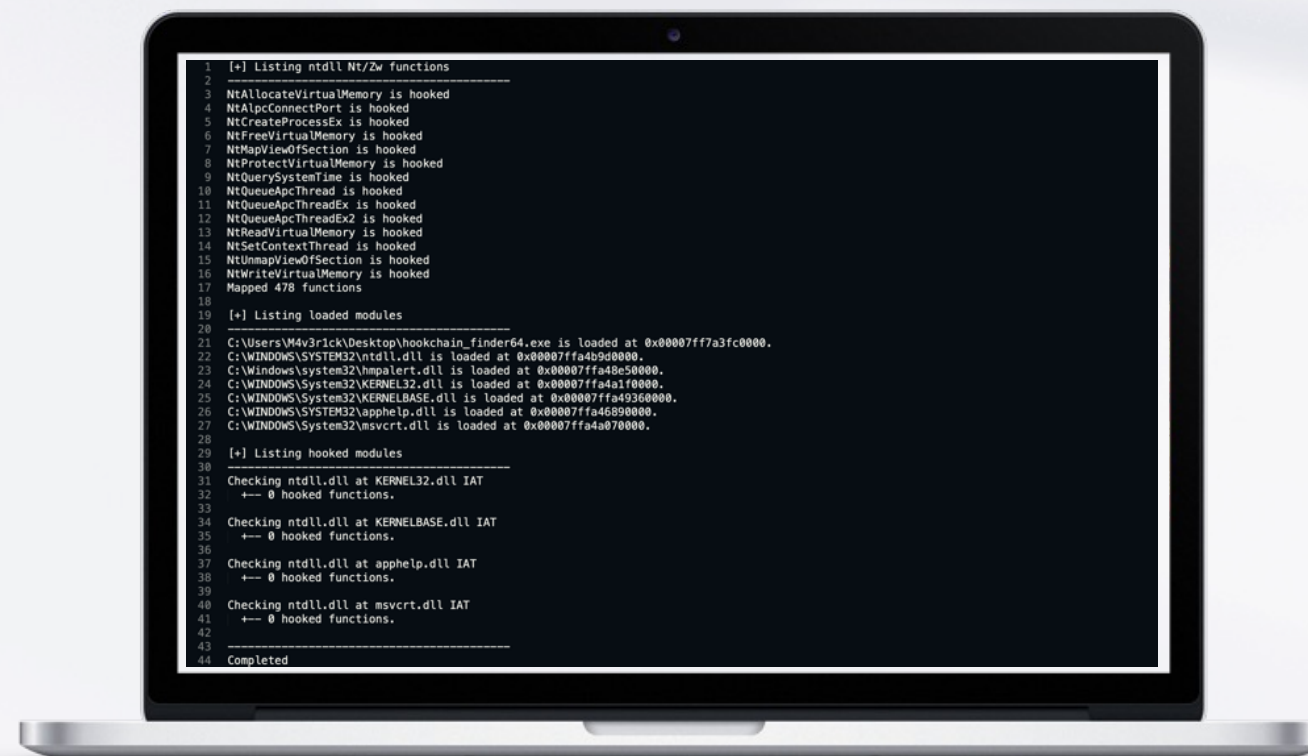
Hooked!
Calculated => 8

HookChain - Viability analysis

Preliminary feasibility, effectiveness, and scope verification of the HookChain technique

Code to just check if there are:

- Ntdll hooks
- IAT hooks



```

1  [+] Listing ntdll Nt/Zw functions
2  -----
3  NtAllocateVirtualMemory is hooked
4  NtAlpcConnectPort is hooked
5  NtCreateProcessEx is hooked
6  NtFreeVirtualMemory is hooked
7  NtMapViewOfSection is hooked
8  NtProtectVirtualMemory is hooked
9  NtQuerySystemTime is hooked
10 NtQueueApcThread is hooked
11 NtQueueApcThreadEx is hooked
12 NtQueueThreadEx2 is hooked
13 NtReadVirtualMemory is hooked
14 NtSetContextThread is hooked
15 NtUnmapViewOfSection is hooked
16 NtWriteVirtualMemory is hooked
17 Mapped 478 functions
18
19 [+] Listing loaded modules
20 -----
21 C:\Users\M4v3rick\Desktop\hookchain_finder64.exe is loaded at 0x00007ff7a3fc0000.
22 C:\WINDOWS\SYSTEM32\ntdll.dll is loaded at 0x00007ffa4b9d0000.
23 C:\Windows\system32\hmpalart.dll is loaded at 0x00007ffa48e50000.
24 C:\WINDOWS\System32\KERNEL32.dll is loaded at 0x00007ffa4a1f0000.
25 C:\WINDOWS\System32\KERNELBASE.dll is loaded at 0x00007ffa49360000.
26 C:\WINDOWS\SYSTEM32\apphelp.dll is loaded at 0x00007ffa46890000.
27 C:\WINDOWS\System32\msvcrt.dll is loaded at 0x00007ffa4a070000.
28
29 [+] Listing hooked modules
30 -----
31 Checking ntdll.dll at KERNEL32.dll IAT
32   ← 0 hooked functions.
33
34 Checking ntdll.dll at KERNELBASE.dll IAT
35   ← 0 hooked functions.
36
37 Checking ntdll.dll at apphelp.dll IAT
38   ← 0 hooked functions.
39
40 Checking ntdll.dll at msvcrt.dll IAT
41   ← 0 hooked functions.
42
43 -----
44 Completed
  
```


Viability analysis

Result 1: 94% of the analyzed EDR solutions (15 out of 16) do not present hooks in the subsystem layer above Ntdll.dll, meaning, in the verification of all DLLs loaded in the application that reference Ntdll, only one EDR solution showed a hook in the IAT.

Result 2: 50% of the analyzed EDR solutions (8 out of 16) show an absence of hooks in user mode.

PRODUCT	INTERCEPTION POINT (HOOK)	
	NTDLL	KERNELBASE / KERNEL32
BitDefender	✓	⊖
CarbonBlack	✓	⊖
Checkpoint	✓	⊖
Cortex	⊖	⊖
CrowdStrike Falcon	✓	⊖
Windows Defender	⊖	⊖
Windows Defender + ATP	⊖	⊖
Elastic	⊖	⊖
ESET	⊖	⊖
Kaspersky	⊖	⊖
MalwareBytes	⊖	⊖
SentinelOne	✓	✓
Sophos	✓	⊖
Symantec	⊖	⊖
Trellix	✓	⊖
Trend	✓	⊖

System Service Number
(SSN)
dynamic mapping

Create a relationship
table with **SSN** +
Custom stub function of
critical Ntdll and
Hooked functions

Implant **IAT Hook** at all
DLL subsystems that
points to **Ntdll** critical
and hooked functions

Continue to desired
actions as usual



HookChain implant flow

1. SSN - System Service Number mapping

Use of one of the dynamic mapping techniques of the SSN presented previously, such as Halo's gate.

HookChain implant flow

1. SSN - System Service Number mapping
2. Map critical functions

Mapping of some base functions for use in the actions of the next steps, such as:

- a. NtAllocateReserveObject
- b. NtAllocateVirtualMemory
- c. NtQueryInformationProcess
- d. NtProtectVirtualMemory
- e. NtReadVirtualMemory
- f. NtWriteVirtualMemory

HookChain implant flow

1. SSN - System Service Number mapping
2. Map critical functions
3. Create and populate SYSCALL_LIST array

Creation and filling of an array where each item contains the following content:

- a. SSN (Syscall Number)
- b. Function address in Ntdll.dll
- c. Memory address of the nearest SYSCALL instruction to the function in Ntdll.dll.

HookChain implant flow

1. SSN - System Service Number mapping
2. Map critical functions
3. Create and populate SYSCALL_LIST array
4. DLL pre-loading

Preloads other DLLs, if it is known that the application in execution will dynamically load and use another DLL that has not yet been loaded in the current process, as well as verifying that this DLL to be loaded makes calls to functions of the Ntdll.dll.

HookChain implant flow

1. SSN - System Service Number mapping

2. Map critical functions

3. Create and populate SYSCALL_LIST array

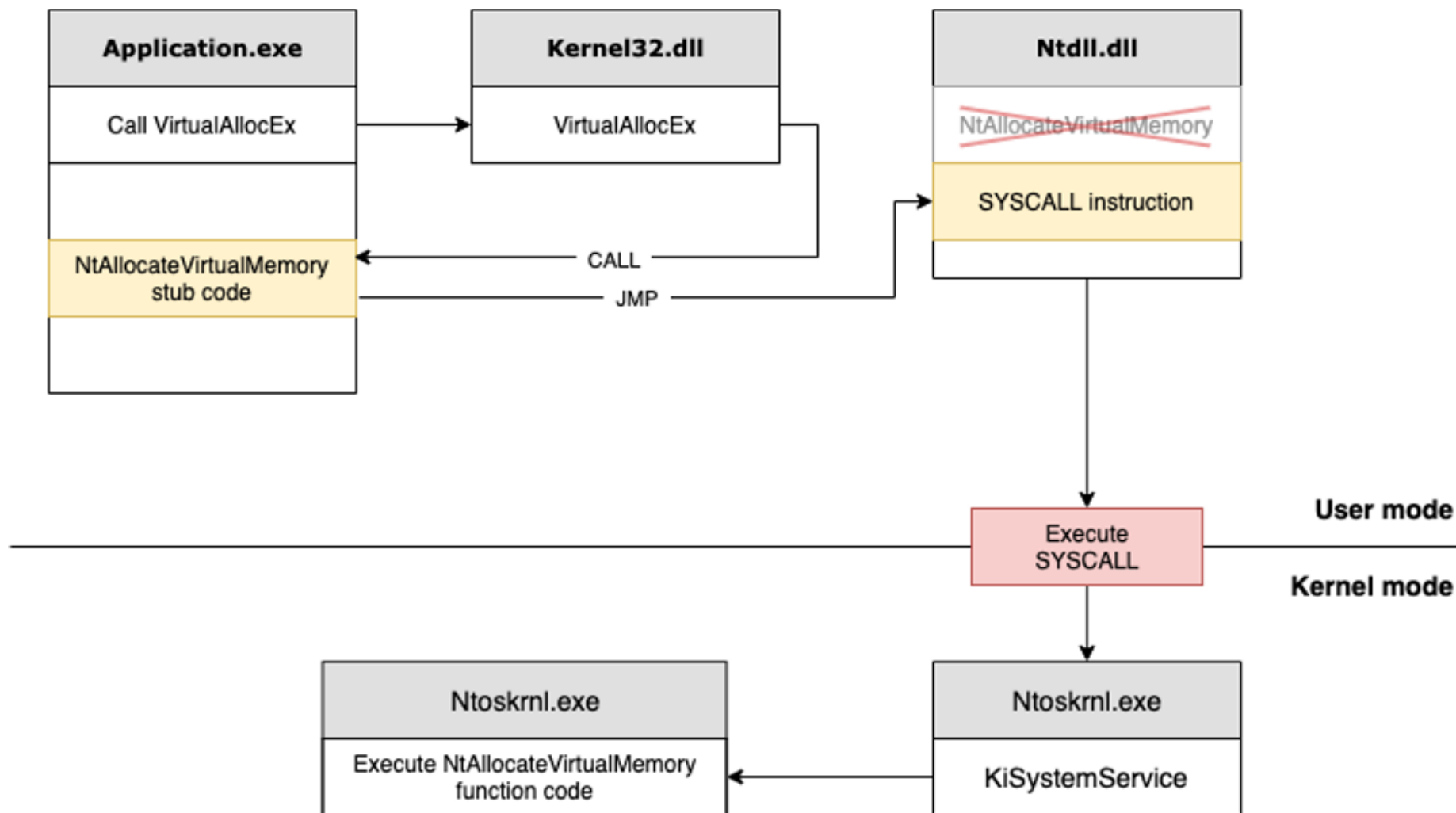
4. DLL pre-loading

5. Implant IAT Hook

- Utilization of Indirect Syscall + mapped critical functions
 - Enumeration, reading and handling of the structures of the export and import tables of loaded DLLs.
- Modification of the IAT of key DLLs that use calls to Ntdll.dll such as:
 - kernel32, kernelbase, bcrypt, bcryptPrimitives, gdi32, mswsock, netutils, and urlmon.

Execution Flow After HookChain Implant

Simplified version



Data structures and tables

Struct SYSCALL_INFO

```
typedef struct _SYSCALL_INFO {
    DWORD64 dwSsn;
    PVOID pAddress;
    PVOID pSyscallRet;
    PVOID pStubFunction;
    DWORD64 dwHash;
} SYSCALL_INFO, * PSYSCALL_INFO;
```

dwSsn: System Service Number.

pAddress: Virtual address of the function within Ntdll.

pSyscallRet: Virtual address of a SYSCALL instruction within Ntdll.

pStubFunction: Virtual address of the HookChain interception (implant) function, this is the address to which all calls to the function in question will be directed. In other words, this is the address that will be assigned in the IAT in replacement of the virtual address of the Ntdll function.

dwHash: Function identification hash. This hash is calculated through the name of the ntdll function. The function name is not stored and used to make identification by EDRs more difficult.

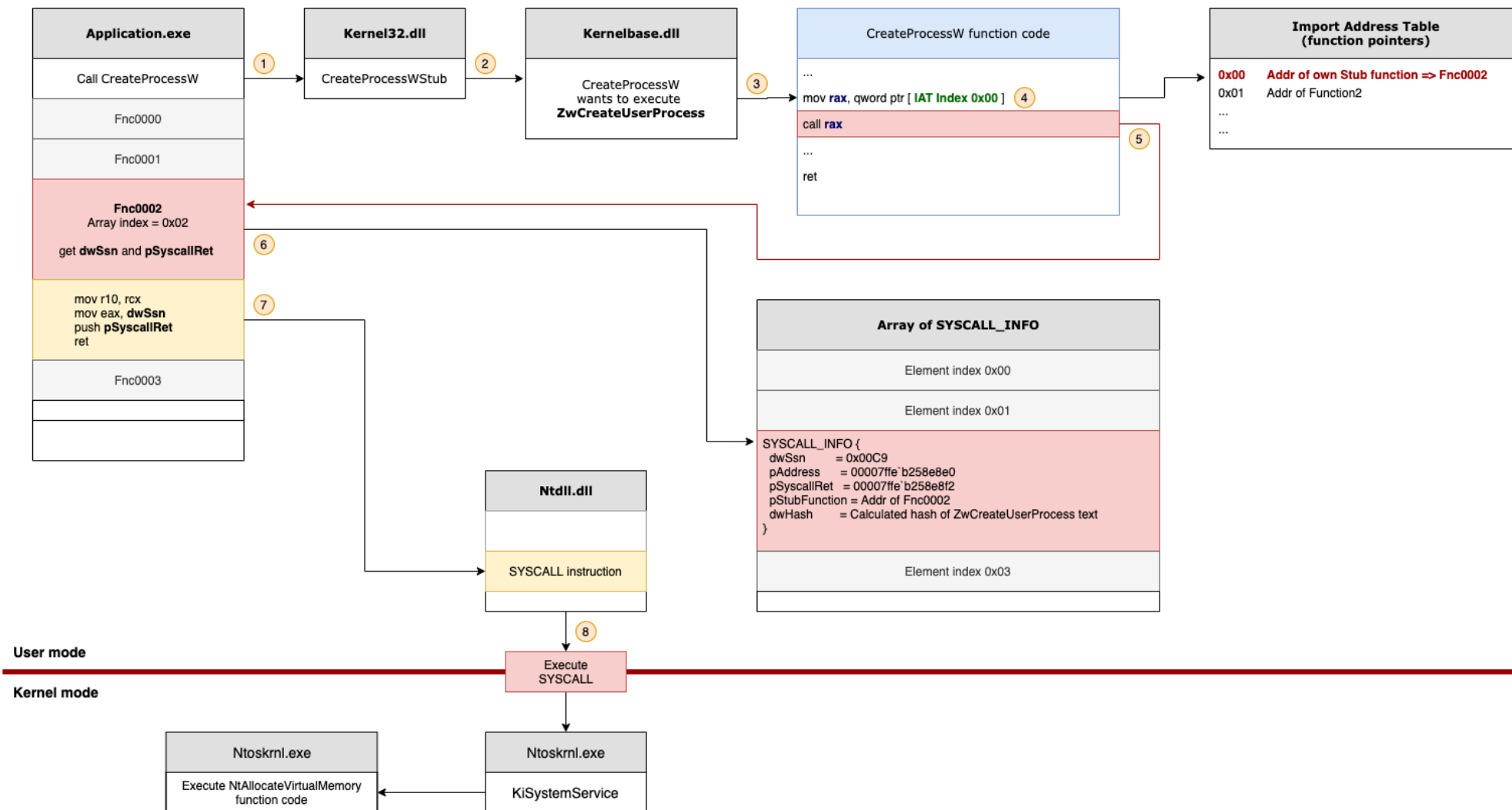
Data structures and tables

Struct SYSCALL_LIST

```
typedef struct _SYSCALL_LIST
{
    DWORD64 Count;
    SYSCALL_INFO Entries[MAX_ENTRIES];
} SYSCALL_LIST, * PSYSCALL_LIST;
```

Array of SYSCALL_INFO

Execution Flow After HookChain Implant



Demo 1

Understanding the flow in a debug environment



Advantages of HookChain



Reduction in the probability of identification by the EDR due to following the rules proposed by some telemetries such as:



Total execution time of the process, as the execution time of the calls will remain very close to the original process.



Execution chain, where the EDR expects the function call to have come from the application, then passed through Kernel32.dll, then through Ntdll.dll. This is due to the fact that the HookChain implant (interception function) passes transparently in the call stack.



No effort and/or modification necessary for the execution of pre-existing codes/applications as the interception occurs broadly and transparently for the application in execution.

Demo 2

Non admin C2 (Command and Control) session



Tests environment!

- 100% of tests had done using the same procedure in my own controlled environment.
- As the test focus is EDR/XDR protections the windows protections (RunAsPPL and Credential Guard) was disabled during the tests.
- Source of Virtual Machines and/or Licensing:
 - Bought by my own resources;
 - Trial licensing;
 - Provided by friends from his corporate environment;

Many thanks to my friends who supported me during this research. Without you, this would not have been possible.

Demo 3

Execute Mimikatz



Gartner Magic Quadrant - 2023

Endpoint Protection Platforms

I was able to test **56%** (9 out of 16) EDR/XDR products present in Gartner 2023 report.

In additional, I tested other 4 products not present in Gartner report, Acronis, Cylance, Elastic and MalwareBytes.

Tested products marked in green.



Result table

PRODUCT	EXECUTED CODE						
	Remote Process Injection	Download, local PE injection and executing					
		Meterpreter	Havoc	Meterpreter + Kiwi	Mimikatz	Procdump	LSASS Dump
Acronis	⚠️	✅	✅	✅	✅	✅	✅
BitDefender	✅	❌	✅	❌	⚠️	❌	⚠️
Cortex	❌	❌	✅	❌	❌	❌	✅
CrowdStrike Falcon	✅	✅	✅	❌	⚠️	⚠️	✅
Cylance	❌	⚠️	❌	❌	✅	❌	❌
Windows Defender	✅	⚠️	✅	❌	❌	❌	✅
Windows Defender XDR	✅	✅	✅	❌	❌	❌	✅
Elastic	✅	❌	❌	❌	❌	❌	❌
ESET	✅	⚠️	✅	✅	✅	❌	✅
MalwareBytes	✅	✅	✅	✅	✅	✅	✅
SentinelOne	⚠️	✅	⚠️	✅	✅	⚠️	⚠️
Sophos	✅	✅	✅	⚠️	⚠️	❌	⚠️
Trellix	✅	✅	✅	✅	✅	❌	✅
Trend	✅	✅	✅	✅	✅	❌	✅

✅ Executed without alerts and blocks.

⚠️ Partially execution (no success) without alerts or Executed (successfully) with alerts.

❌ Execution fail with block and alert

I'm just a child who has never grown up. I still keep asking these 'how' & 'why' questions. Occasionally, I find an answer.

Stephen Hawking



<https://github.com/helviojunior/hookchain/>

- Full white paper
- Source code
- References

